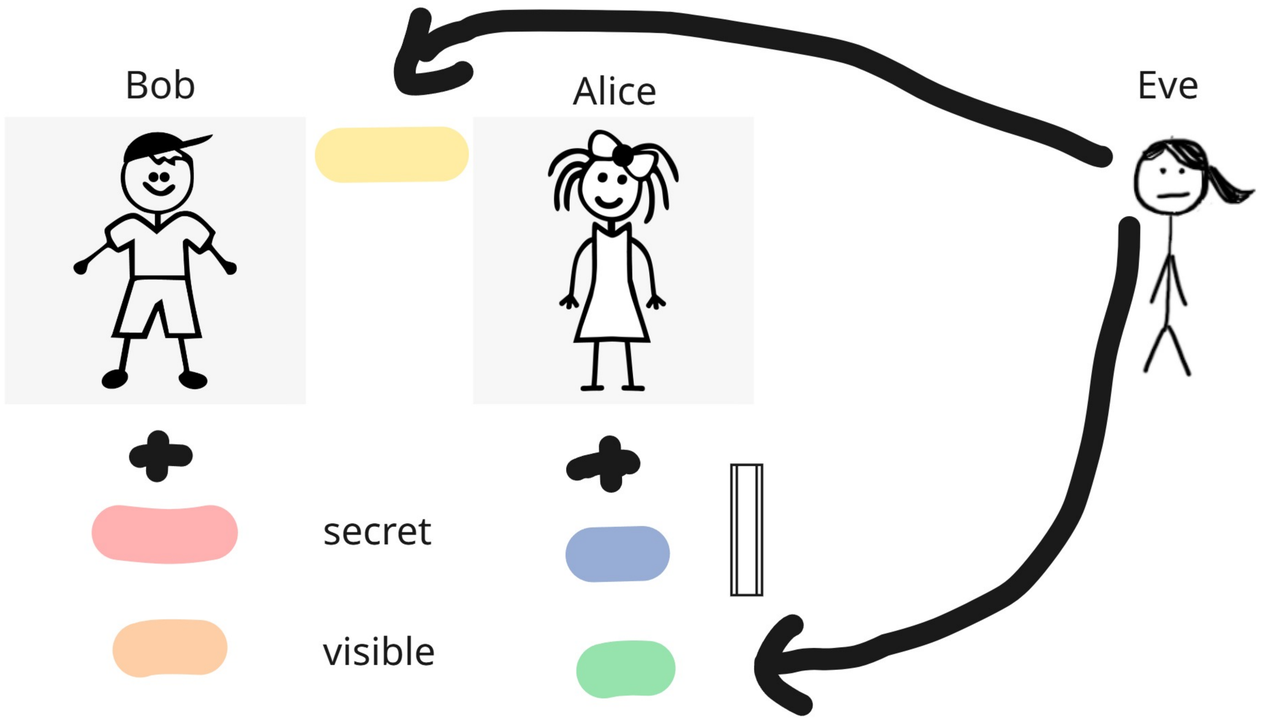
Diffie-Hellman Key Exchange

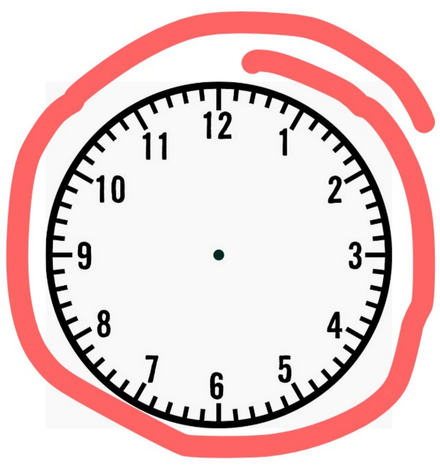
The Dilemma: How do two people establish a secret password over a network where an eavesdropper (Eve) is logging every single packet? If you just send the password, Eve gets it. If you encrypt the password, how do you share the key to decrypt it?

Scenario

Modular Arithmetic



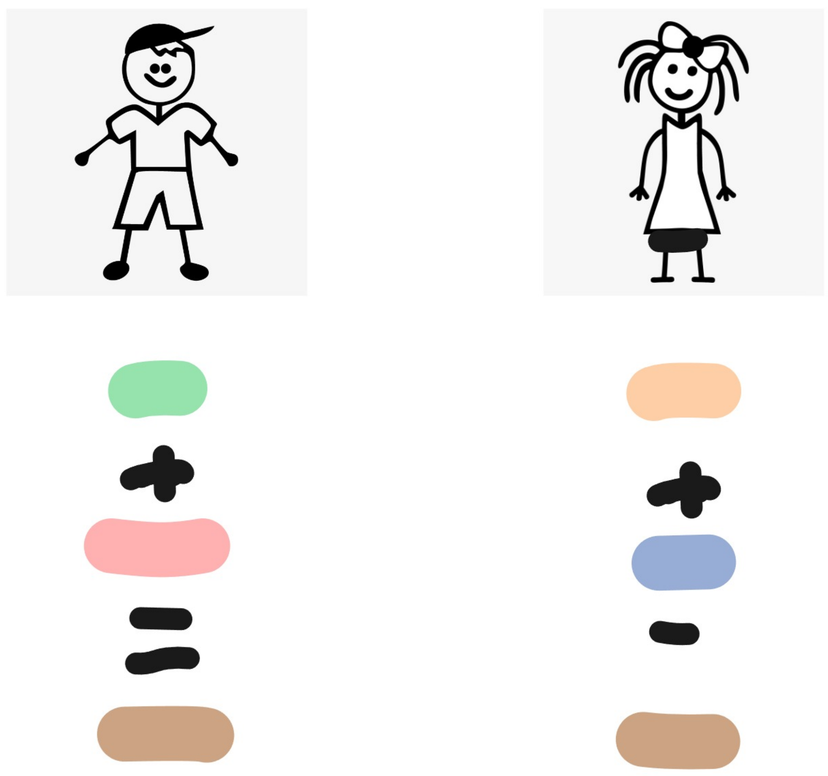
14 (mod 12) = 2



16 (mod 12) = ? 4

5 (mod 3) = ? 2

9 (mod 3) = ? 0



Discrete Logarithmic Problem

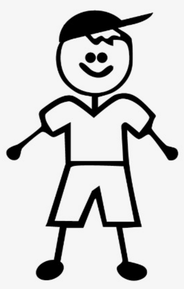
Why this is a one-way street: If I tell you, *"I started at a certain hour, added a bunch of hours, wrapped around the clock a few times, and landed on 2,"* can you guess my exact starting number?

No. It could have been 1, 5, 9 etc. **The wrapping process destroys the history of how you got there.** This is what prevents an eavesdropper from reversing the math.

The Ingredients

A massive prime number (p) (public)
A generator (g) -> creates randomness

Bob



Alice

b = 6 (secret)

a = 4

$$B = g^b \pmod p$$

$$A = g^a \pmod p$$

Let's use **p = 17** and **g = 3**.

B = 3^6 (mod 17)

A = 3^4 (mod 17)

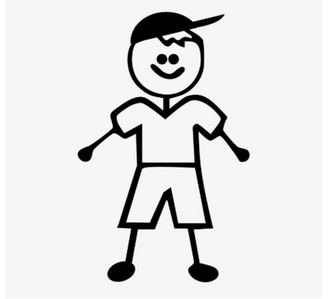
B = 15

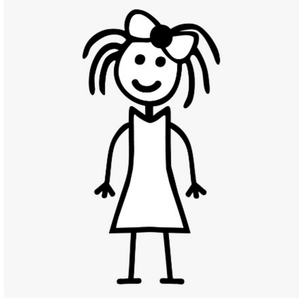
A = 13

Eve

Eve is sniffing the network.
She intercepts: p = 17, g = 3, A = 13, and B = 15

To steal the secret, she has to solve: 3^a (mod17) = 13. With tiny numbers, she can guess. With a 2048-bit prime, her computer will burn out before she finds it.





B = 13^6 (mod 17)

A = 15^4 (mod 17)

B = 16

A = 16

The reason they get the exact same number boils down to a high-school exponent rule: (x^y)^z = x^yz

Both generated the number 16 without ever sharing it. They can now use 16 as the password for an AES-encrypted chat.

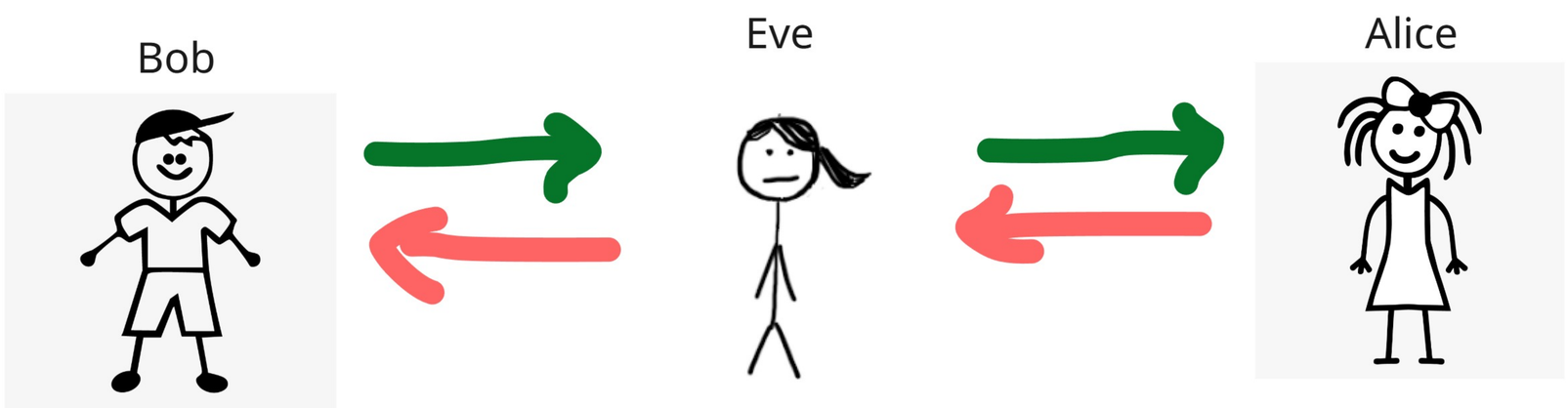
Modern Context

Perfect Forward Secrecy (PFS): Instead of using static keys, a brand-new key pair is generated for *every single session*. If a server's master private key is stolen a year from now, past traffic remains completely unreadable.

ECDH (Elliptic-Curve Diffie-Hellman): Standard DH requires massive keys (e.g., 2048-bit or 3072-bit) to stay secure, whereas ECDH achieves the same security with much smaller keys (e.g., 256-bit), saving massive amounts of compute and bandwidth.

The Flaw

Standard Diffie-Hellman does not provide authentication. Alice thinks she is exchanging keys with Bob, but she might actually be exchanging keys with Eve, who is simultaneously exchanging keys with Bob.



The Fix

DH must be paired with an authentication mechanism, like digital signatures or SSL/TLS certificates (which is exactly where RSA or ECDSA comes back into the picture to sign the DH parameters)

And that is the magic of the Diffie-Hellman key exchange. It is a brilliant piece of mathematical choreography that allows two complete strangers to establish a **bulletproof secret key over a completely compromised network**, without ever actually sharing the key itself. It’s the unsung hero keeping your daily internet traffic secure. But remember—Diffie-Hellman only lets you agree on a secret; it doesn't actually prove *who* you're talking to. To solve that identity crisis, we need digital signatures and full asymmetric encryption. And for that, we have to look at **RSA**. If the clock math finally clicked for you today, hit that subscribe button, drop a comment below, and I’ll see you in the next video where we break down RSA.